Redes (TA048)

Facultad de Ingeniería — Universidad de Buenos Aires

Trabajo Práctico Nº 2

SDN: NAT con POX y Mininet

NAT saliente con traducción por puertos (PAT)

2do cuatrimestre 2026
Comisión jueves

Integrantes:

Alen Monti — 108081

Nadeska Millan — 110779

Elías Cuba — 106933

Netanel Jamilis — 99093

Sebastian Furnier — 110129

6 de noviembre de 2026

1. Introducción

El presente trabajo práctico tiene como objetivo implementar un **NAT saliente con traducción por puertos (PAT, *Port Address Translation*)** en el contexto de *Redes Definidas por Software* (SDN).

A diferencia de una arquitectura tradicional, en SDN el comportamiento de la red no está distribuido en los dispositivos de forwarding, sino que es definido por un **controlador** que se comunica con ellos mediante el protocolo **OpenFlow**. Esto separa el *plano de control* (decisiones de ruteo/traducción, en el controlador) del *plano de datos* (reenvío de paquetes, en los switches).

La solución se desarrolla sobre el controlador **POX** y se valida con la herramienta de emulación **Mininet**. El controlador provisto por la cátedra (`protorouter.py`) implementaba solamente: procesamiento de paquetes IPv4, detección de tráfico de la red privada, reescritura de MACs e instalación de reglas de forwarding. Sobre esa base se extendió el código para implementar:

- **Protocolo ARP dinámico**: respuesta de requests dirigidas al router, generación de requests para resolver MACs y mantenimiento de una tabla ARP con expiración.
- **NAT/PAT**: traducción de IP y puerto origen (TCP/UDP) de las conexiones iniciadas desde la red privada, con mantenimiento de estado y soporte para múltiples hosts y conexiones simultáneas que comparten una única IP pública.
- **Instalación de flujos**: por cada conexión se instalan reglas en ambos sentidos para que el switch realice la traducción y el forwarding sin intervención posterior del controlador, con timeouts para liberar recursos.

2. Descripción general de la solución

2.1. Topología

La topología extendida (`topo.py`) consta de un switch OpenFlow (`s1`) con tres hosts conectados:
- `h1`: host en la red pública (servidor), `200.0.0.1/24`, gateway `200.0.0.254`.
- `h2`: host privado, `192.168.1.2/24`, gateway `192.168.1.254`.
- `h3`: segundo host privado, `192.168.1.3/24`, mismo gateway.

El controlador hace que `s1` actúe como un router L3 con dos interfaces:

Interfaz	IP	MAC
Pública (puerto 1)	`200.0.0.254`	`00:00:00:aa:aa:aa`
Privada (puertos 2-3)	`192.168.1.254`	`00:00:00:bb:bb:bb`

Tabla 1: Interfaces del router implementado sobre el switch s1.

Importante: **no existe ninguna entrada ARP estática** en ningún host; además, ninguna MAC o IP de los hosts está hardcodeada en el controlador. La dirección del servidor puede cambiarse (IP y/o MAC) y la solución sigue funcionando, porque todo se descubre dinámicamente por ARP.

2.2. Arquitectura del controlador

El módulo `ext/protorouter.py` mantiene, por cada conexión de switch:
- una **tabla ARP dinámica** (`IP -> MAC`, con expiración a los 5 minutos);
- una **cola de resoluciones pendientes**: paquetes que no se pudieron reenviar porque falta la MAC destino y se enviarán cuando llegue la respuesta ARP;
- la **tabla NAT**, con un registro por conexión, indexada en ambos sentidos (clave saliente y clave entrante) para demuxear en $O(1)$;
- un **conjunto de puertos públicos en uso** para evitar colisiones;
- un **timer** periódico (10 s) que expira entradas ARP viejas y registros NAT cerrados.

El flujo de un paquete en el `PacketIn` es:

1. Si es **ARP**: se aprende de la tupla (IP, MAC) que traiga y se responde o inunda (*flood*) según corresponda (sección 3).
2. Si es **IPv4**:
   - origen privado y destino público ⇒ *tráfico saliente*: se busca/crea el registro NAT, se traduce IP y puerto de origen, se resuelve la MAC destino por ARP si hace falta y se instala el par de flujos (sección 4).
   - destino = IP pública del router ⇒ *tráfico entrante*: se demuxea por la clave entrante de la tabla NAT, se traduce IP y puerto de destino hacia el host privado original y se reenvía por la interfaz privada.
   - cualquier otra combinación se descarta.

3. Manejo de ARP

El protocolo ARP es el componente que permite al NAT resolver direcciones MAC en ambas redes, sin entradas estáticas. La implementación tiene tres partes.

3.1. Respuesta de requests dirigidas al router

Cuando llega un ARP request cuyo `protodst` es una de las IPs del router (`200.0.0.254` o `192.168.1.254`), el controlador responde inmediatamente con un ARP *reply* que usa la MAC de la interfaz correspondiente (`aa:aa:aa...` para la pública, `bb:bb:bb...` para la privada). Así, cualquier host que quiera comunicarse con "su gateway" la resuelve dinámicamente.

3.2. Generación de requests

Cuando el NAT necesita reenviar un paquete y no conoce la MAC destino, envía un ARP request desde la interfaz por donde saldría el paquete (con la IP y MAC de esa interfaz como origen) y **enchufa** (*queue*) el paquete. Al llegar el reply, se aprende la MAC, se envían todos los paquetes encolados y se completan las instalaciones de flujo que quedaron frenadas. Esto se necesita tanto para el servidor público (`h1`) como para los hosts privados (`h2`, `h3`).

3.3. Tabla ARP dinámica y aprendizaje

Todo paquete ARP (request o reply) que atraviesa el switch se usa para *aprender*: la tupla (`protosrc`, `hwsrc`) se guarda en la tabla junto con el timestamp y el puerto del switch por el que se aprendió. Las entradas expiran a los 5 minutos (`ARP_ENTRY_TIMEOUT`), y un timer periódico hace la limpieza. Si llega una MAC distinta para una IP conocida, se actualiza la entrada (y se lo indica en el log).

Los ARP requests dirigidos a otras IPs (por ejemplo, entre hosts del mismo segmento) se inundan por el switch como en un aprendizaje convencional.

4. Diseño de la tabla NAT (PAT)

4.1. Registro por conexión

Cada conexión traducida se representa con la clase `NATRecord`:

Campo	Significado
`proto`	Protocolo: 6 (TCP) o 17 (UDP)
`private_ip`	IP del host interno
`private_port`	Puerto origen del host interno
`remote_ip`	IP del host público (servidor)
`remote_port`	Puerto destino en el servidor
`public_port`	Puerto público asignado en el NAT

Tabla 2: Campos de un registro de la tabla NAT.

La tabla se indexa en ambos sentidos:

- **clave saliente** ($proto$, IP_{priv}, $port_{priv}$, IP_{pub}, $port_{pub}$): para detectar si un paquete saliente pertenece a una conexión ya traducida;
- **clave entrante** ($proto$, $port_{pblico}$, IP_{pub}, $port_{pub}$): para demuxear los paquetes que llegan del servidor y saber a qué host privado redirigirlos.

Además, cada registro guarda un timestamp y expira tras `NAT_RECORD_TIMEOUT` (65 s, levemente mayor al `hard_timeout` de los flujos) para que no queden "colgados" registros de conexiones que ya no tienen flujos en el switch.

4.2. Ejemplo conceptual

Protocolo	IP privada	Puerto priv.	IP pública	Puerto púb.
TCP	192.168.1.2	42480	200.0.0.254	42480
TCP	192.168.1.3	50000	200.0.0.254	50000
TCP	192.168.1.2	42482	200.0.0.254	51000
UDP	192.168.1.3	5353	200.0.0.254	5353

Tabla 3: Ejemplo de la tabla NAT con conexiones simultáneas de h2 y h3.

Obsérvese que dos hosts distintos pueden incluso compartir el mismo puerto público a la vez si van a servidores distintos: la clave entrante incluye la IP del servidor, así que no hay ambigüedad.

4.3. Asignación de puertos públicos

La función `_pick_public_port` implementa la política de asignación:

1. Si el puerto origen privado cae en el rango asignable $[10000, 65535]$ y no está en uso para ese protocolo, **se reutiliza** (evita traducciones innecesarias).
2. Si no, se asigna un puerto del mismo rango elegido al azar, avanzando cíclicamente hasta encontrar uno libre. La elección aleatoria del punto de partida distribuye la carga sobre el espacio de puertos.
3. El par $(proto, puerto)$ se marca como usado; se libera cuando el registro expira.

Con esto se garantiza que dos conexiones simultáneas nunca obtengan el mismo ($proto$, $puerto_pblico$) y que los recursos se reutilicen cuando las conexiones terminan.

4.4. Traducción de paquetes

Para el tráfico saliente se reescriben: IP origen (→ `PUBLIC_IP`), puerto origen TCP/UDP (→ puerto público asignado) y MAC origen (→ `PUBLIC_MAC`); la MAC destino se resuelve por ARP.

Para el entrante, simétricamente: IP y puerto destino (→ host privado original) y MAC destino del host privado.

Los checksums de IP y de la capa de transporte se recalculan al reempaquetar el paquete con la librería de POX (`pack()`), que recompute el checksum de IP y de TCP/UDP con los nuevos valores de los campos.

4.5. ICMP

El ping (ICMP echo) no tiene puertos, pero sí un *identificador* de echo que cumple la misma función de demultiplexado. El NAT lo trata de forma análoga: usa el identificador como "puerto" para crear el registro, traduce la IP origen y demuxea los echo replies entrantes por esa clave. Así, `h2 ping h1` funciona y puede coexistir con los pings de `h3`.

5. Estrategia de instalación de flujos

El objetivo del diseño es que **el switch haga todo el trabajo** una vez establecida la traducción, minimizando la intervención del controlador.

5.1. Cuándo se instalan

Los flujos de una conexión se instalan en el primer paquete de la conexión (cuando se crea el registro), pero **solo cuando se conocen las dos MACs** involucradas (la del servidor y la del host privado). Si alguna falta, se dispara la resolución ARP y la instalación se completa al llegar la respuesta. Esto evita instalar reglas con MACs incorrectas o vacías.

5.2. Reglas instaladas

Por cada conexión se envían dos `ofp_flow_mod` con *match exacto*:

- **Saliente**: `in_port` = puerto privado donde entró el primer paquete, `dl_type`=0x800, `nw_src`, `nw_dst`, `nw_proto`, `tp_src` y `tp_dst` de la conexión. Acciones: `SET_NW_SRC` (IP pública), `SET_TP_SRC` (puerto público, si cambia), `SET_DL_SRC/DST` (MACs) y `OUTPUT` por el puerto público.
- **Entrante**: `in_port` = puerto público, `nw_src/nw_dst/tp_src/tp_dst` del flujo invertido visto desde el router. Acciones: `SET_NW_DST` (IP privada), `SET_TP_DST` (puerto privado, si cambia), reescritura de MACs y `OUTPUT` por el puerto privado del host.

Como el *match* es exacto (incluye puertos), las reglas de distintas conexiones no colisionan y la tabla del switch no crece con entradas agregadas.

5.3. Timeouts y liberación de recursos

Cada flujo se instala con:

- `idle_timeout = 10` s: si no pasa tráfico, la regla se elimina;
- `hard_timeout = 60` s: límite máximo de vida de la regla;
- flag `OFPFF_SEND_FLOW_REM`: el switch notifica al controlador la remoción, lo que permite (en futuras extensiones) liberar el puerto público en ese momento.

En paralelo, el registro NAT expira a los 65 s si no hay actividad, y el timer periódico libera el puerto público. De esta forma, tanto las reglas del switch como el estado del controlador se limpian solos.

5.4. Tráfico que llega al controlador

Con este diseño, el controlador solo ve: (a) paquetes ARP, (b) el primer paquete de cada conexión, (c) paquetes cuya MAC destino todavía no se resolvió y (d) tráfico cuya conexión ya expiró (que re-dispara la creación del registro). Todo el tráfico de datos de una conexión establecida circula únicamente por el switch.

6. Decisiones de diseño

1. **Match exacto por conexión** en lugar de reglas agregadas por host: evita ambigüedades al demuxear el tráfico entrante y hace trivial la liberación de recursos por expiración. El costo es una entrada más en la tabla de flujos por conexión, aceptable para la escala del TP.

2. **Doble indexación** de la tabla NAT (clave saliente y entrante apuntando al mismo registro): el demuxeo del tráfico entrante, que es la operación crítica, es $O(1)$ sin recorrer la tabla.

3. **Cola de paquetes por resolución ARP**: en lugar de descartar el paquete que disparó el ARP request (lo que perdería el SYN de una conexión TCP, por ejemplo), se encola y se envía al llegar el reply. Esto hace que la primera conexión no pierda paquetes.

4. **ICMP tratado como un "flujo con puerto"**: el identificador de echo hace de puerto, lo que permite que los pings de varios hosts privados convivan sin colisiones, con el mismo mecanismo de PAT.

5. **Reutilización del puerto origen** cuando es posible: reduce la cantidad de traducciones (solo se reescribe el puerto si cambió) y simplifica el debugging, porque el puerto visto en el servidor es el mismo que el del cliente.

6. **No hardcodear nada de los hosts**: tanto las MACs de los hosts privados como la del servidor se descubren por ARP, y la IP del servidor se aprende del propio tráfico. Cambiar IP/MAC de cualquier host no rompe la solución (requisito de la demo).

7. Problemas encontrados y soluciones

Respuestas que llegan antes que el flujo entrante.
Cuando el primer paquete saliente genera el ARP request hacia el servidor, el reply del servidor puede llegar antes de que el flujo entrante esté instalado. Solución: el registro NAT se crea *antes* de instalar los flujos, así que aunque el paquete entre por `PacketIn` ya se puede demuxear; la instalación de flujos se completa al resolver la MAC.

Colisiones de puertos entre hosts.
Dos hosts privados pueden usar el mismo puerto origen a la vez. El conjunto de puertos en uso se lleva por (*proto*, *puerto*), de modo que el segundo pedido recibe otro puerto automáticamente.

Checksums.
Cambiar IP o puerto invalida el checksum de IP y el de TCP/UDP. Se resuelve reempaquetando el paquete con la librería de POX, que recalcula ambos checksums con los valores finales.

ARP estático de la topología original.
La topología provista configuraba entradas ARP estáticas en los hosts para el debugging inicial. Se eliminaron por completo y se verificó que la primera comunicación de cada host resuelve su gateway por ARP dinámico.

Flujos con MACs desconocidas.
Instalar un flujo con la MAC destino aún sin resolver provocaba tráfico hacia MACs inválidas. Los flujos se instalan recién cuando se conocen ambas MACs; mientras tanto, el controlador procesa los paquetes de esa conexión.

8. Pruebas realizadas

8.1. Conectividad básica

Con el controlador y la topología corriendo:

```
mininet> h2 ping -c 3 200.0.0.1
mininet> h3 ping -c 3 200.0.0.1
```

Listing 1: Ping desde ambos hosts privados hacia el servidor.

Ambos pings funcionan. En el log del controlador se observa: (1) el ARP request de cada host resolviendo su gateway, (2) la creación de los registros ICMP en la tabla NAT y (3) la instalación de los flujos, tras lo cual los siguientes paquetes del ping ya no generan `PacketIn`.

8.2. Múltiples clientes

En la ejecución registrada, `h2` y `h3` iniciaron conexiones TCP hacia `200.0.0.1:9000`. El controlador asignó puertos públicos distintos y demuxeó los paquetes de respuesta hacia la IP y el puerto privado correspondientes. La evidencia disponible muestra ambos flujos a través del NAT; no permite afirmar que las transferencias hayan sido simultáneas.

8.3. Diferentes tipos de tráfico

La salida adjunta registra tráfico ICMP echo (ida y vuelta) y TCP. No contiene una prueba UDP, por lo que ese protocolo queda pendiente de verificación en esta ejecución. Para TCP/UDP la traducción opera sobre los campos de puertos, sin inspeccionar el payload.

8.4. Resultados observados en la ejecución

El log de POX confirma el aprendizaje ARP dinámico del servidor `200.0.0.1` y de los hosts privados, seguido por la instalación de flujos. Los mapeos NAT relevantes fueron:

Protocolo	Host privado	Puerto/id privado	Puerto/id público	Destino
ICMP	192.168.1.2	3023	49381	200.0.0.1, secuencia 1
ICMP	192.168.1.3	3029	16179	200.0.0.1, secuencia 1
TCP	192.168.1.2	54892	54892	200.0.0.1:9000
TCP	192.168.1.3	39556	39556	200.0.0.1:9000

Tabla 4: Traducciones observadas en el log de la ejecución compartida.

Para ICMP se observan respuestas entrantes dirigidas de vuelta a `192.168.1.2` y expiración posterior de los dos registros. En TCP, las respuestas del servidor se reenviaron a `192.168.1.2:54892` y `192.168.1.3:39556`, respectivamente. Los puertos TCP privados estaban dentro del rango permitido y libres, por lo que se reutilizaron como puertos públicos. El log de arranque también advierte que POX 0.7.0 declara soporte para Python 3.6–3.9; la ejecución mostrada continuó con Python 3.12.3.

8.5. Pruebas unitarias

Se agregaron pruebas automatizadas para las rutas ICMP echo saliente y entrante con dos hosts, la expiración y liberación de un registro NAT, el aislamiento de los mapeos TCP y sus respuestas, y las entradas ARP de tres campos (MAC, timestamp y puerto), incluyendo su expiración. Se ejecutan desde la raíz del repositorio con:

```
python3 -m unittest tests.unit.protorouter_test
```

La ejecución de este comando completó correctamente las tres pruebas.

8.6. Independencia de valores hardcodeados

Se modificaron la MAC y la IP del servidor (y de los hosts privados) en `topo.py` y la solución siguió funcionando sin cambios en el controlador, ya que todo el estado de MACs se aprende por ARP.

9. Conclusiones

El trabajo permite ver en concreto cómo una función clásica de red (el NAT) puede implementarse como *aplicación* sobre un controlador SDN: la "inteligencia" (crear traducciones, resolver ARP, decidir qué hacer con un paquete nuevo) vive en el plano de control, y el trabajo repetitivo (traducir y reenviar cada paquete de una conexión ya establecida) se delega al plano de datos mediante reglas OpenFlow.

Los puntos que más aportan al aprendizaje son: el demuxeo del tráfico entrante por (*puerto_pblico*, *proto*), que es exactamente lo que hace un NAT real; la necesidad de mantener estado con expiración coherente entre el controlador y los timeouts del switch; y la importancia de ARP como infraestructura sobre la que se apoya todo el resto (sin resolución dinámica de MACs, el NAT no puede reescribir tramas).

Una extensión natural sería liberar el puerto público al recibir el `FlowRemoved` del switch (en lugar de por timer), y aplicar timeouts distintos para TCP (según estado de la conexión) y UDP.